



Ayudantía 5

Búsqueda: BFS y DFS

Por Anaís Montanares y Felipe Espinoza

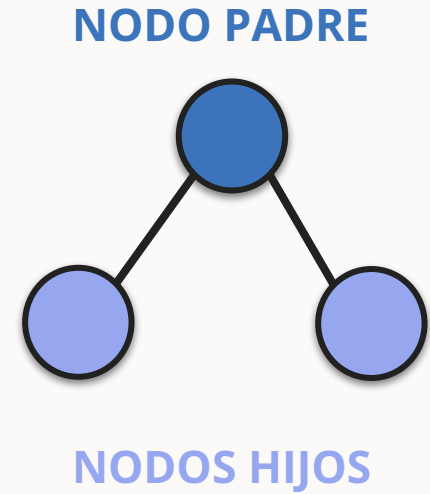
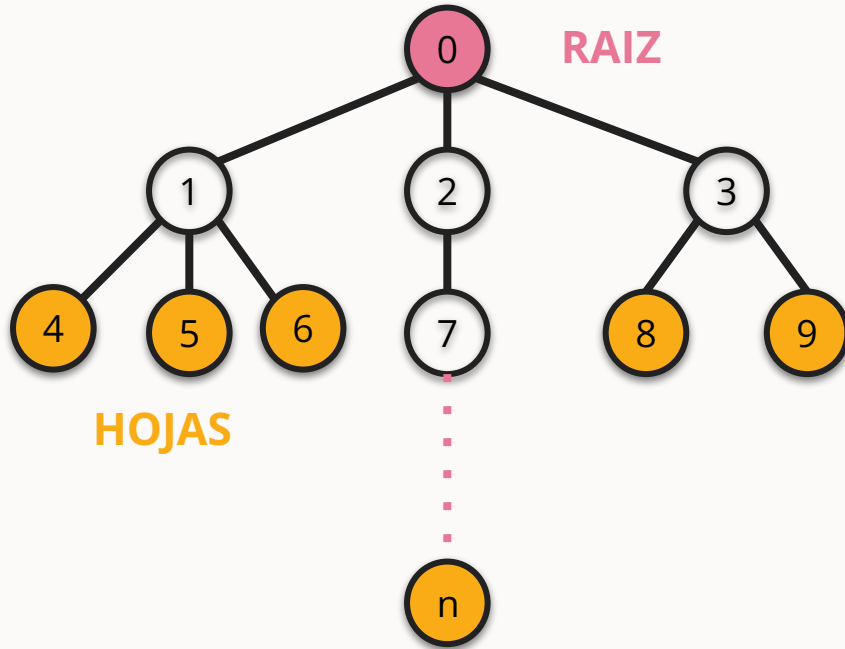
14 de abril 2025



Contenidos

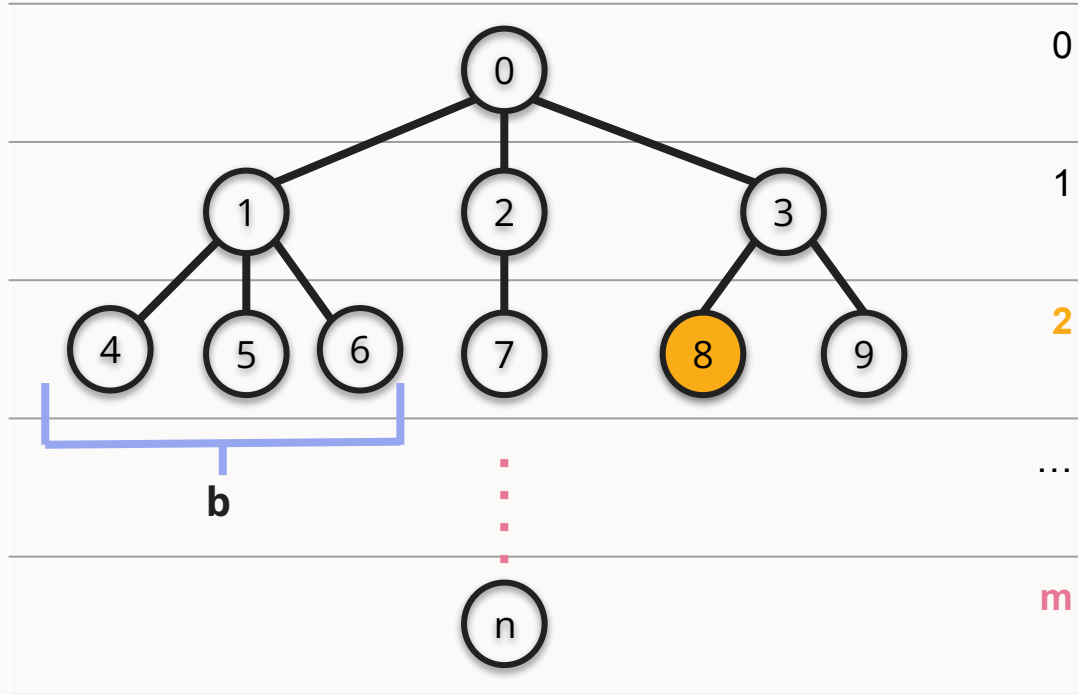
1. Árboles
2. Algoritmos de búsqueda (BFS, DFS y IDDFS)
3. Heurísticas

Árboles





Árboles



b: Factor de ramificación promedio (hijos/padres)

p: Profundidad de la solución

m: Largo de la rama más larga



Algoritmos de Búsqueda



Búsqueda Genérica

- **S:** Conjunto de nodos o estados.
- **A:** Conjunto de acciones o transiciones entre nodos.
- **s_init:** Nodo inicial desde donde se empieza la búsqueda.
- **G:** Conjunto de nodos objetivo o estados meta.



Búsqueda Genérica

Open : contiene todos los nodos **generados** pero no explorados

Closed : contiene los nodos **explorados**

Sucesores : **sucesores** o hijos de un nodo



Búsqueda Genérica

Input: Un problema de búsqueda (S, A, S_{init}, G)

Output: Un nodo objetivo

```
1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta  $S_{init}$  a Open
4   $parent(S_{init}) = \text{null}$ 
5  while Open  $\neq \emptyset$ 
6       $u \leftarrow \text{Extraer}(\text{Open})$ 
7      Inserta  $u$  en Closed
8      for each  $v \in \text{Succ}(u) \setminus (\text{Open} \cup \text{Closed})$ 
9           $parent(v) = u$ 
10         if  $v \in G$  return  $v$ 
11         Inserta  $v$  a Open
```

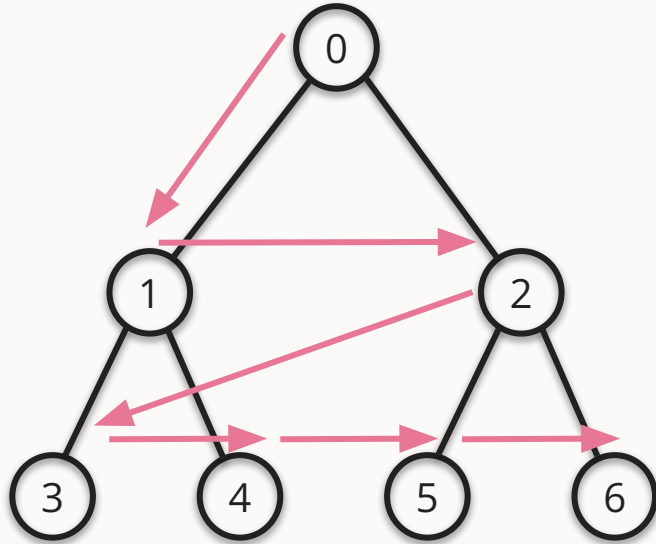
La diferencia con el resto de los algoritmos es la estructura de datos que se usa para *Open*



Algoritmos BFS y DFS

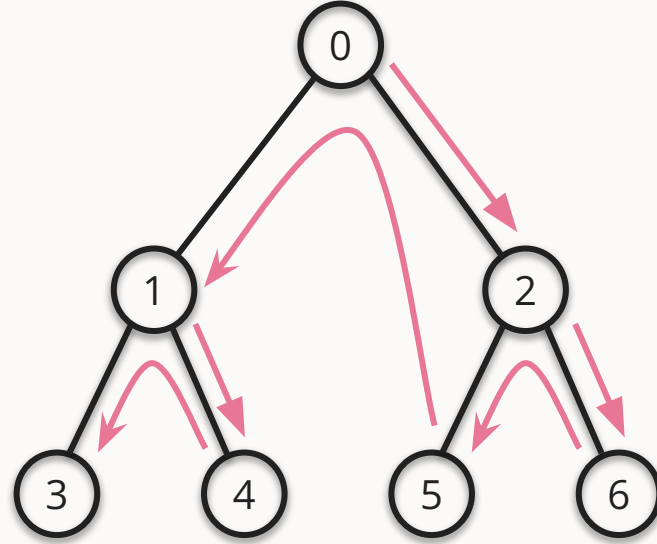
BFS - BREADTH FIRST SEARCH

Búsqueda en amplitud



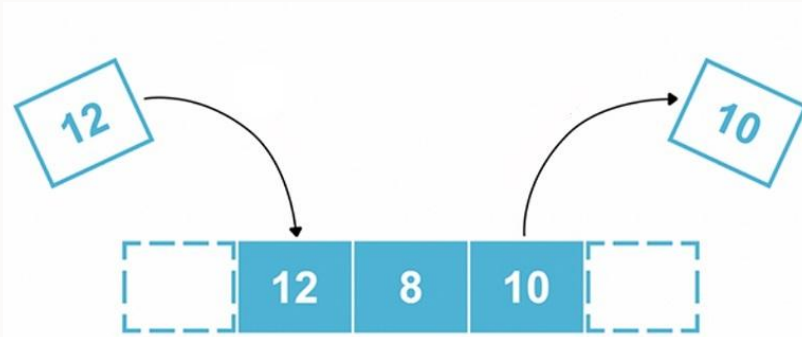
DFS - DEPTH FIRST SEARCH

Búsqueda en profundidad



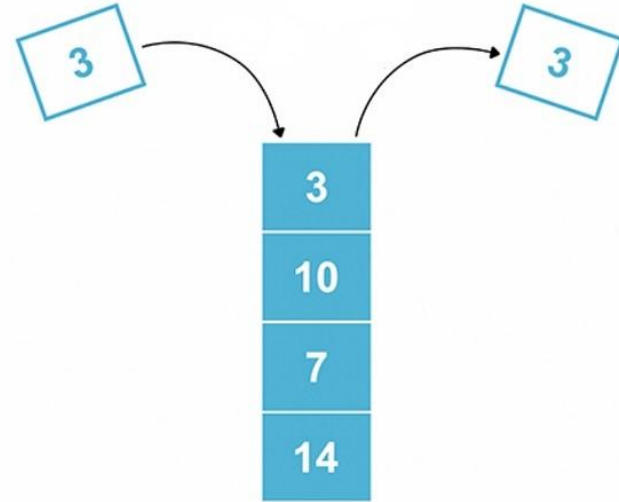


Extraer: BFS VS DFS



Queue

Estructura de *Open* en **BFS**



Stack

Estructura de *Open* en **DFS**



BFS VS DFS

b: Factor de ramificación promedio
p: Profundidad de la solución
m: Largo de la rama más larga

BFS	DFS
Memoria: $O(b^p)$ Tiempo: $O(b^p)$	Memoria: $O(bm)$ Tiempo: $O(b^m)$
Si el espacio de estados es finito, es completo y óptimo para problemas de búsqueda con costos uniformes	Si el espacio de estados es finito, con detección de ciclos es completo

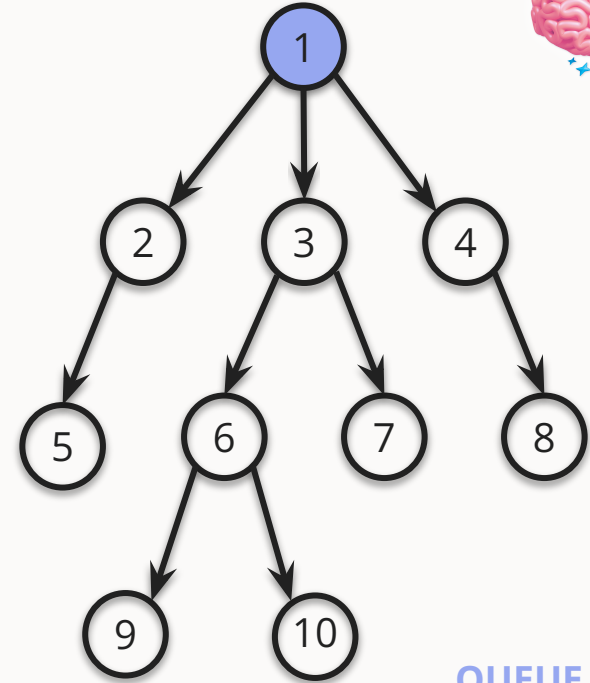


BFS

Input: El árbol de la figura

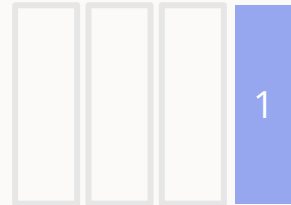
Output: Queremos retornar el nodo 10

- 1 *Open* es un contenedor vacío
- 2 *Closed* es un conjunto vacío
- 3 Inserta *Sinit* a *Open*
- 4 $\text{parent}(Sinit) = \text{null}$
- 5 **while** *Open* $\neq \emptyset$
- 6 $u \leftarrow \text{Extraer}(\text{Open})$
- 7 Inserta u en *Closed*
- 8 **for each** $v \in \text{Succ}(u) \setminus (\text{Open} \cup$
- 9 *Closed*)
- 10 $\text{parent}(v) = u$
- 11 **if** $v \in G$ **return** v
- Inserta v a *Open*



QUEUE

Open: {1}
Closed: {}



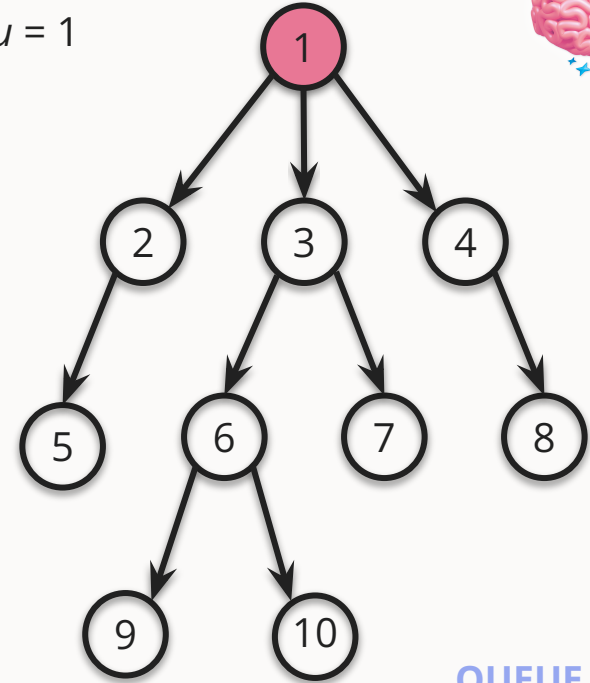
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 1$



QUEUE



Open: {}
Closed: {1}



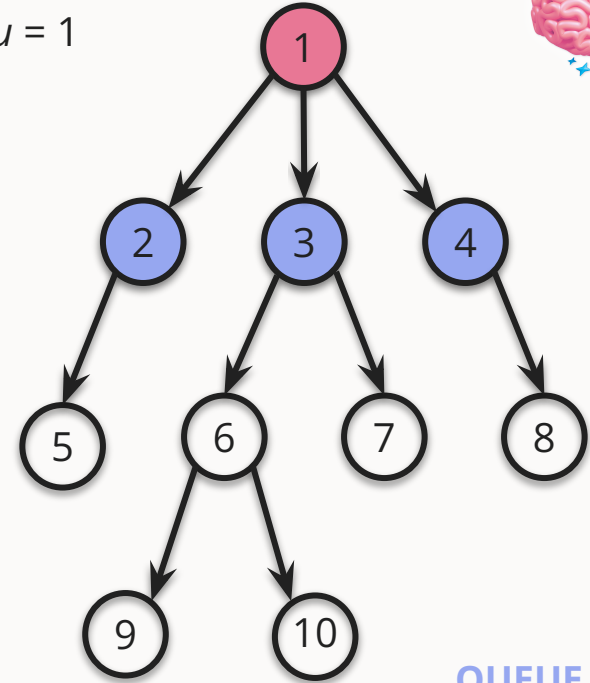
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 1$



QUEUE



Open: {2, 3, 4}
Closed: {1}

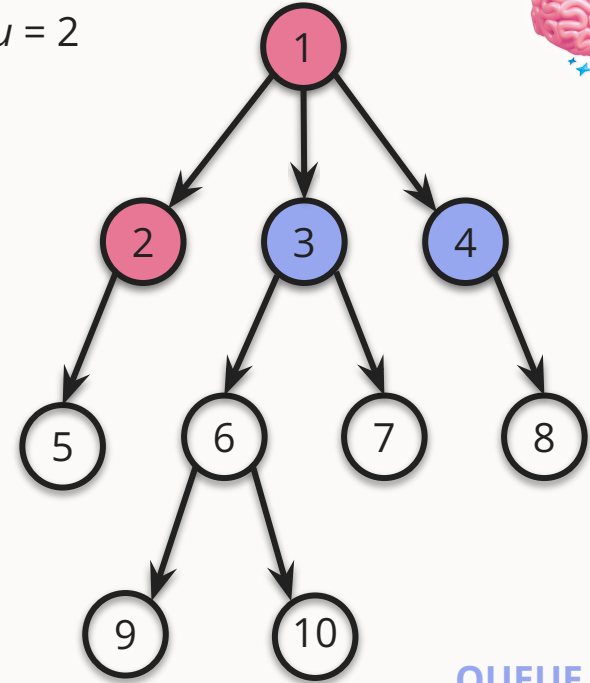
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

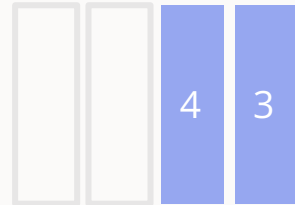
1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 2$



QUEUE

Open: {3, 4}
Closed: {1, 2}



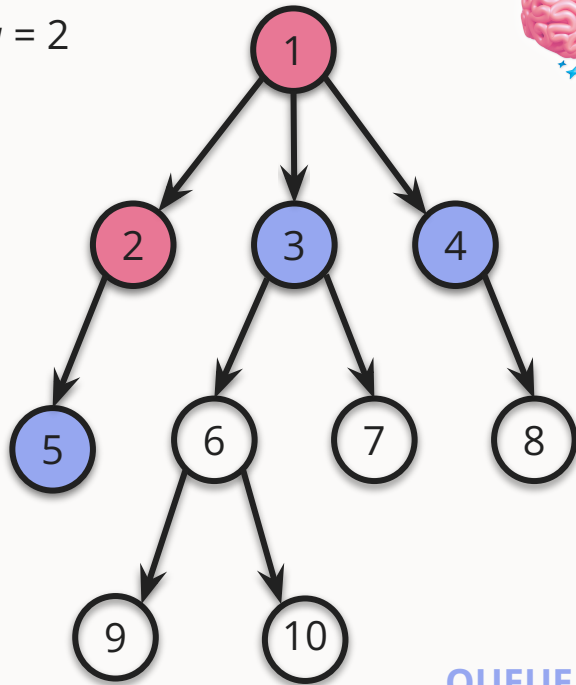
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 2$



QUEUE

Open: {3, 4, 5}
Closed: {1, 2}



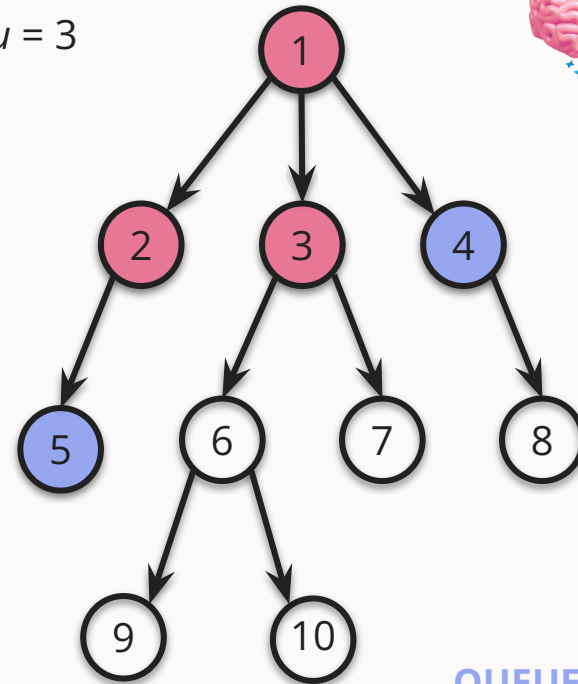
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

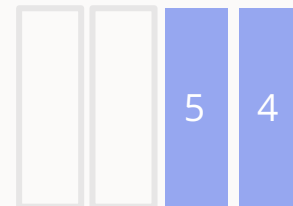
$u = 3$



QUEUE

Open: {4, 5}

Closed: {1, 2, 3}



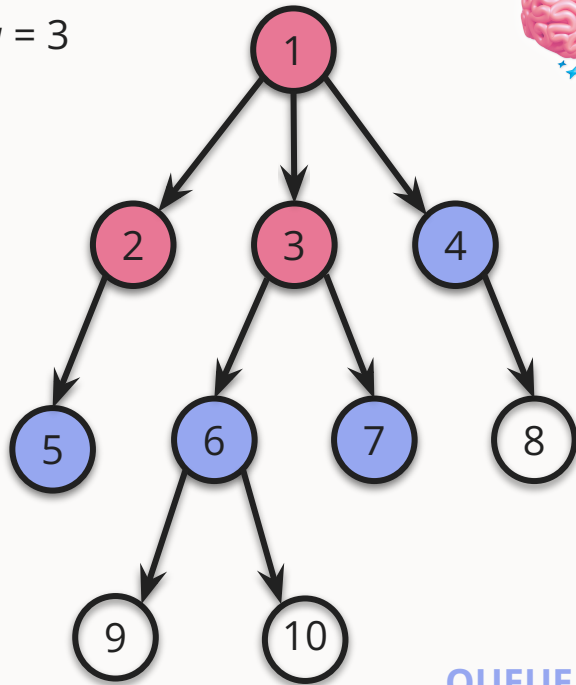
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

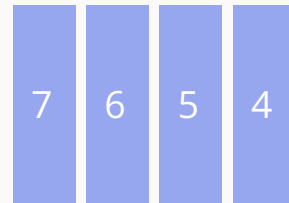
```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 3$



QUEUE



Open: {4, 5, 6, 7}
Closed: {1, 2, 3}

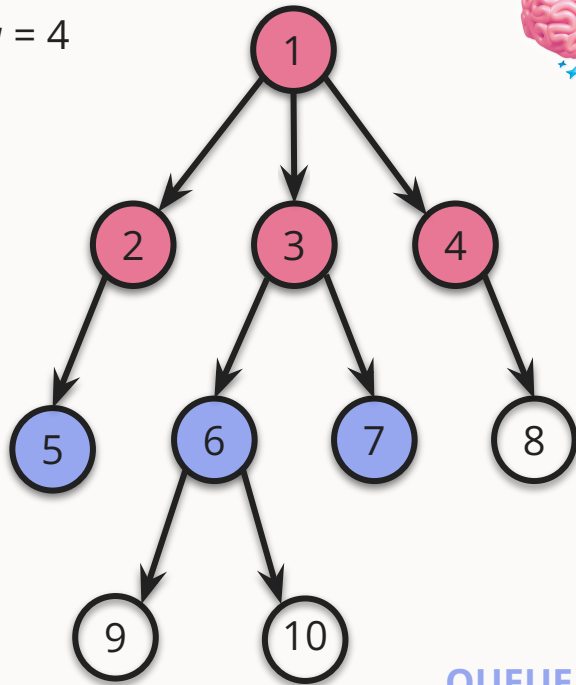
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

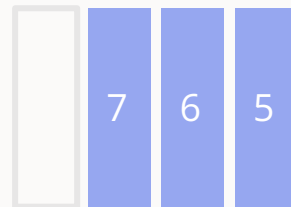
$u = 4$



QUEUE

Open: {5, 6, 7}

Closed: {1, 2, 3, 4}



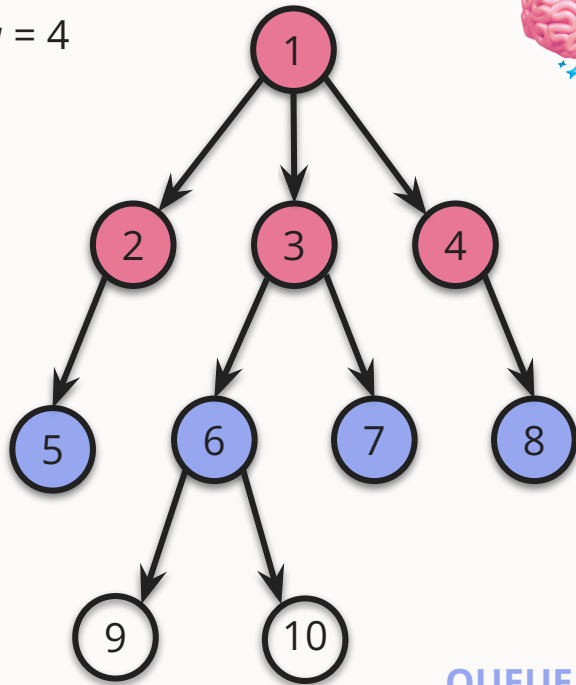
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

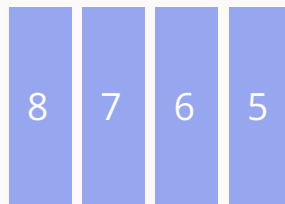
```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 4$



QUEUE



Open: {5, 6, 7, 8}
Closed: {1, 2, 3, 4}

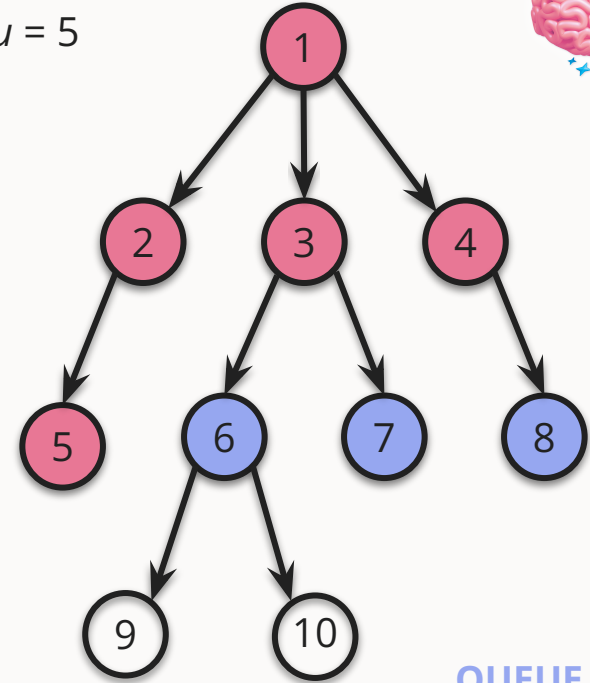
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 5$



QUEUE

Open: {6, 7, 8}

Closed:
{1, 2, 3, 4, 5}



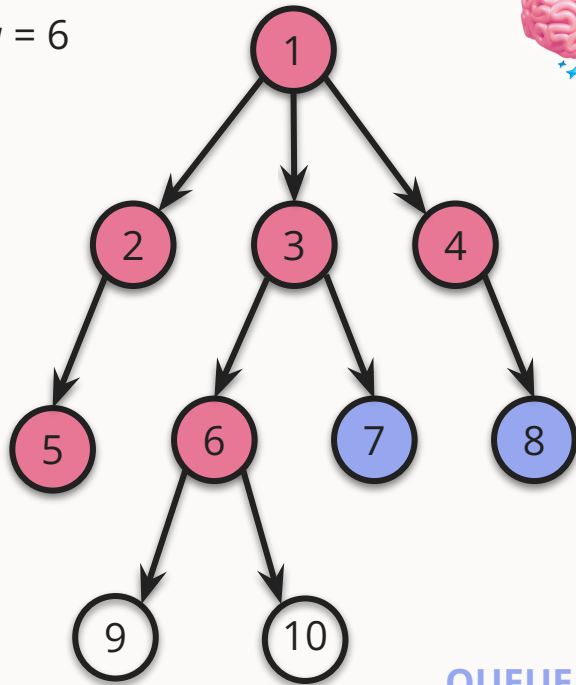
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

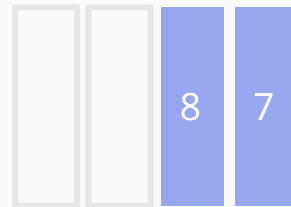
$u = 6$



QUEUE

Open: {7, 8}

Closed:
{1, 2, 3, 4, 5, 6}



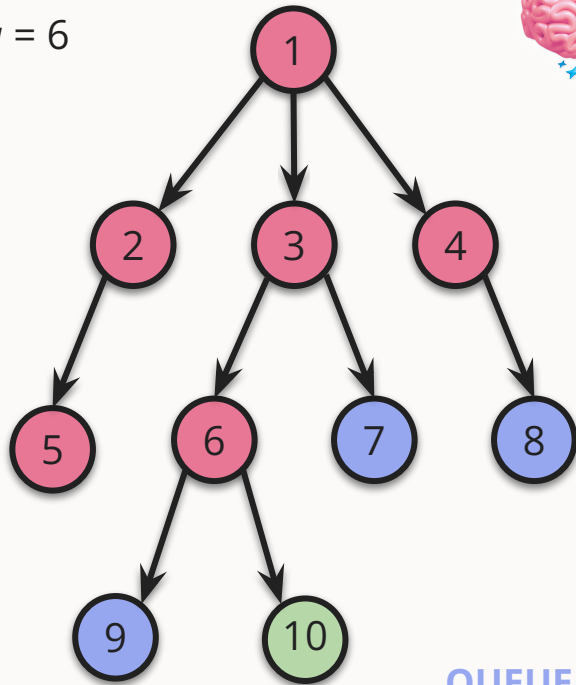
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

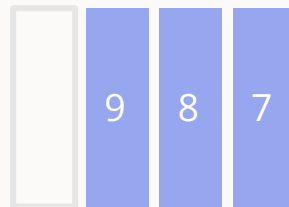
$u = 6$



Open: {7, 8, 9}

Closed:
{1, 2, 3, 4, 5, 6}

QUEUE





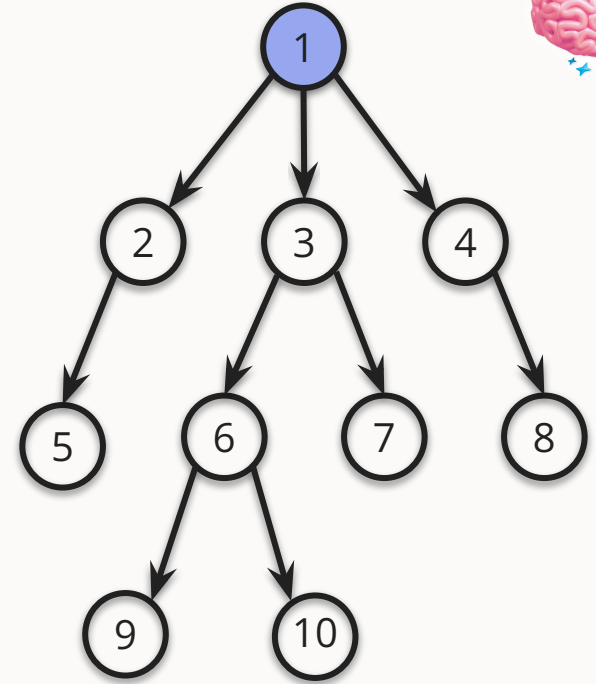
DFS

Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```



Open: {1}
Closed: {}

1
STACK



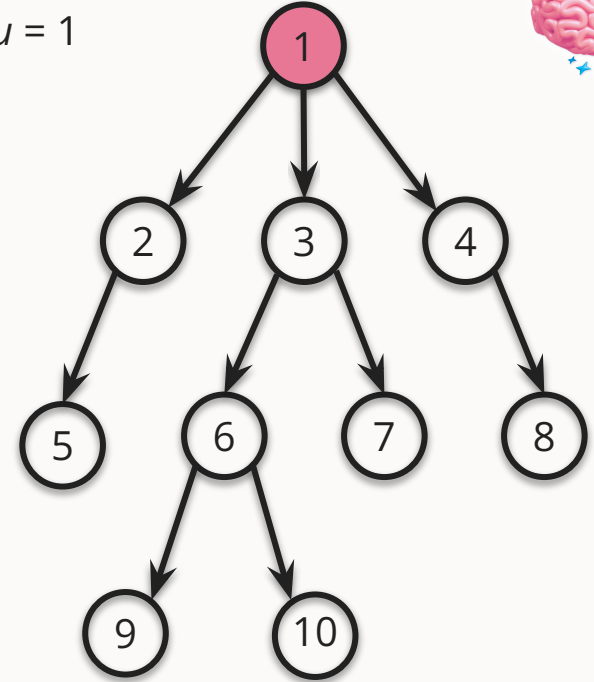
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 1$



Open: {}
Closed: {1}

STACK



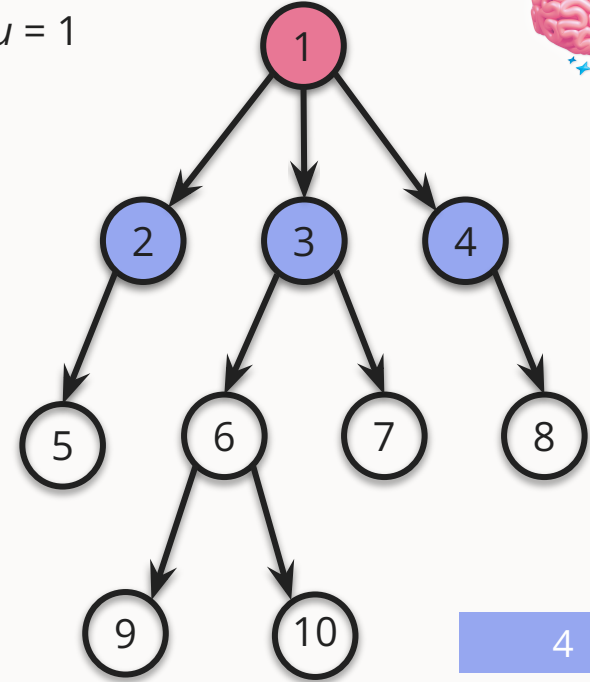
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

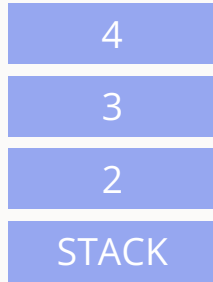
```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 1$



Open: {2, 3, 4}
Closed: {1}



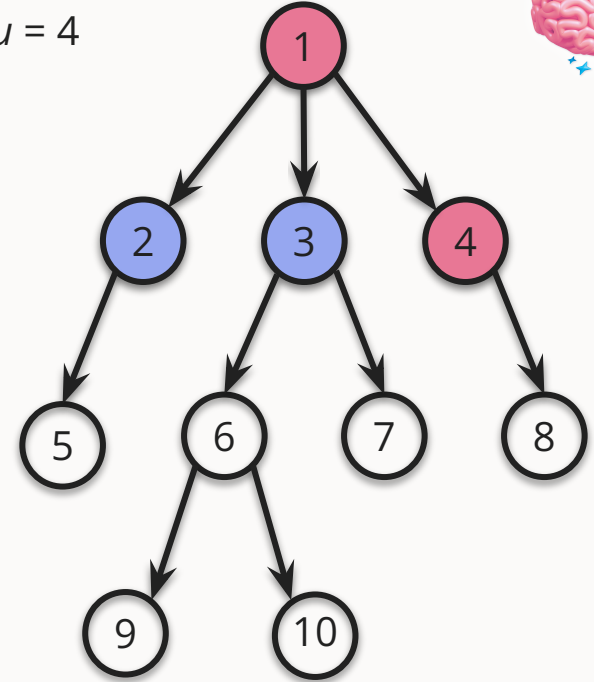
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 4$



Open: {2, 3}
Closed: {1, 4}

3

2

STACK



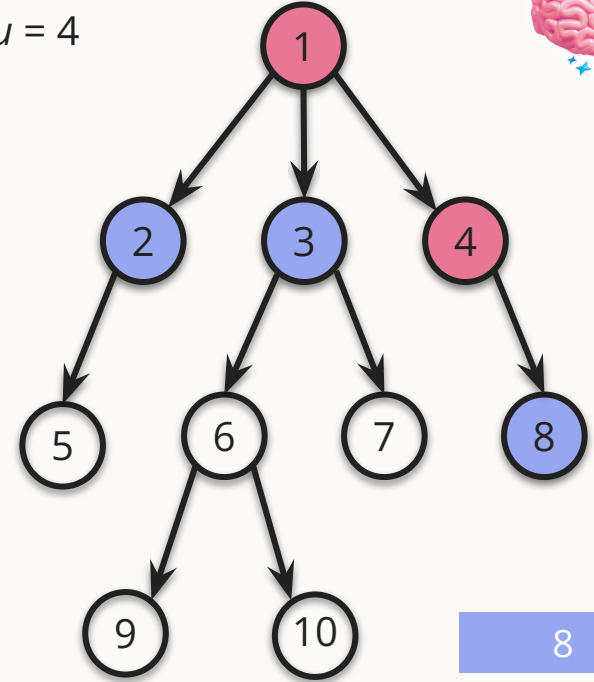
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 4$



Open: {2, 3, 8}
Closed: {1, 4}

8
3
2
STACK



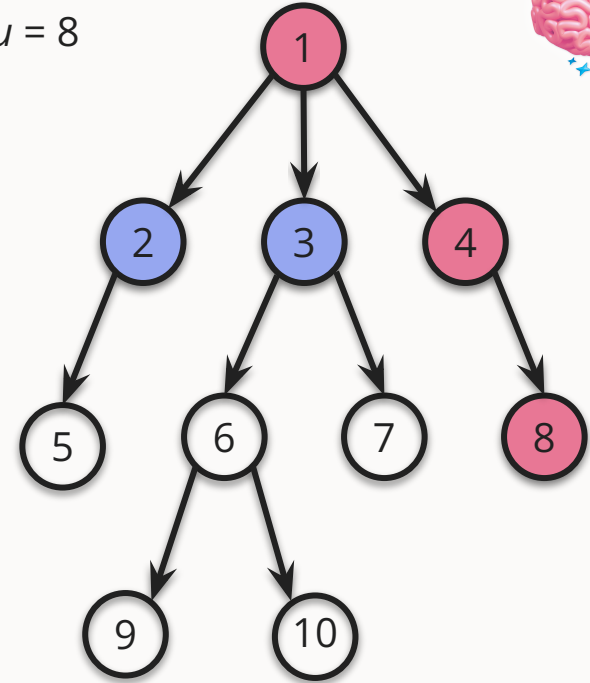
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 8$



Open: {2, 3}

Closed: {1, 4, 8}

3

2

STACK



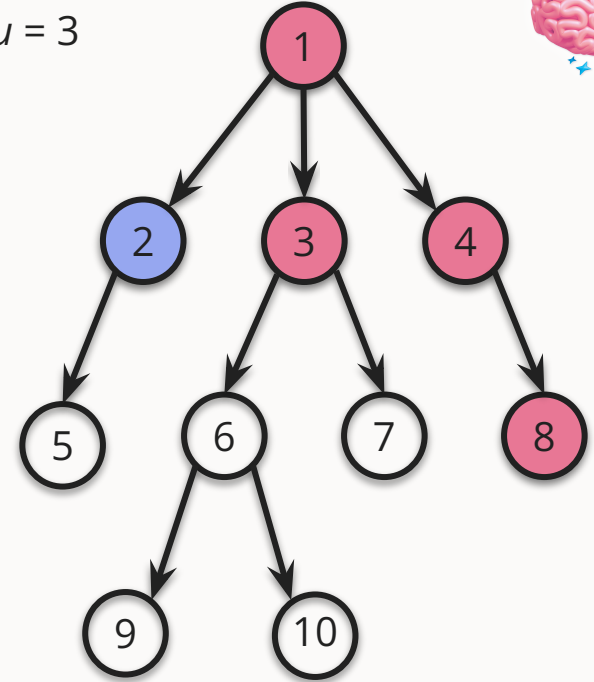
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 3$



Open: {2}

Closed: {1, 4, 8, 3}

2

STACK



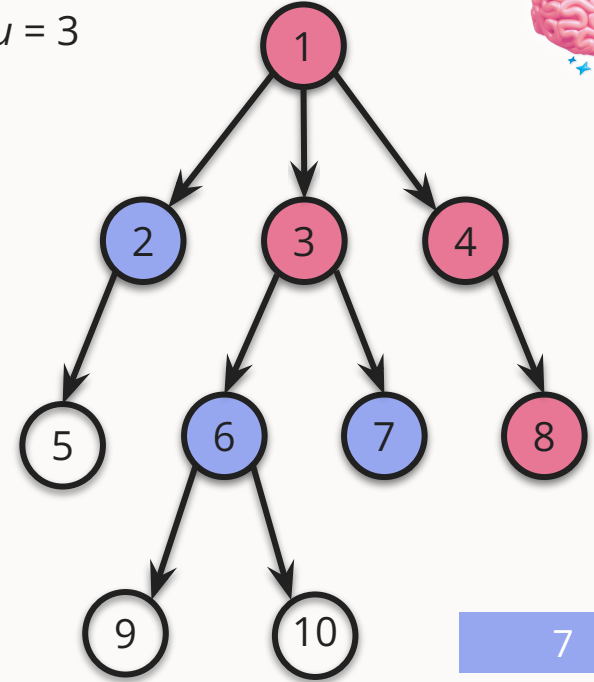
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 3$



Open: {2, 6, 7}

Closed: {1, 4, 8, 3}

7

6

2

STACK

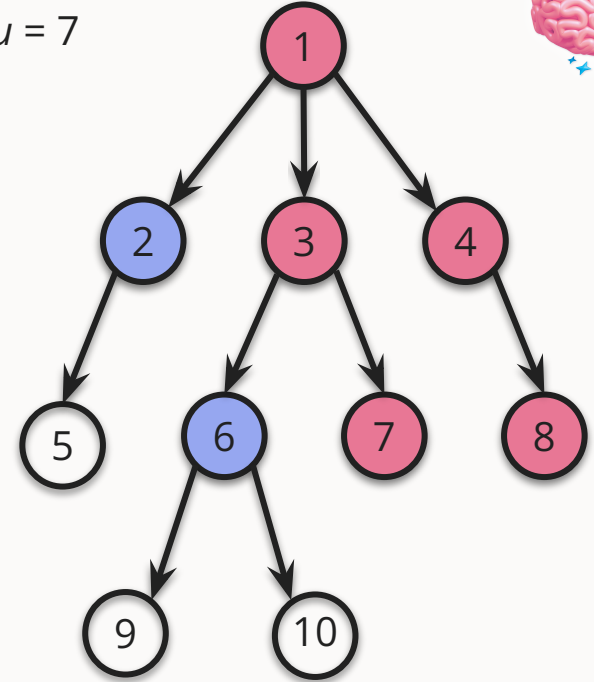
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 7$



Open: {2, 6}

Closed: {1, 4, 8, 3, 7}

6

2

STACK



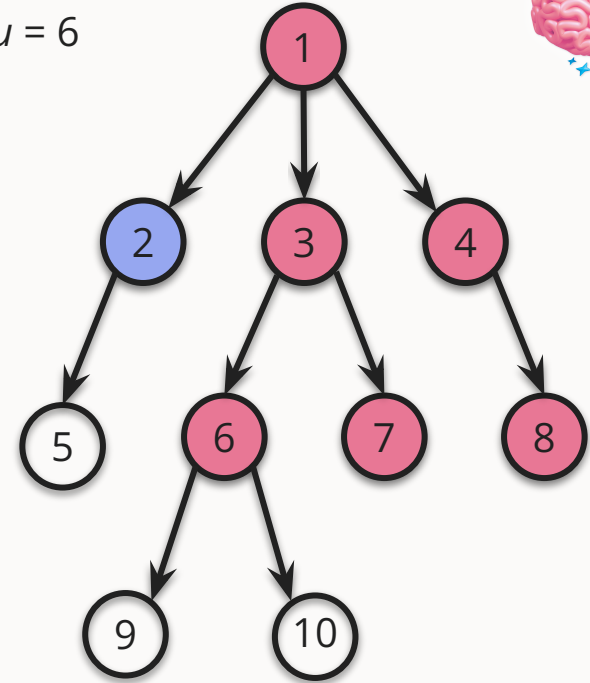
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 6$



Open: {2}

Closed: {1, 4, 8, 3, 7, 6}

2

STACK



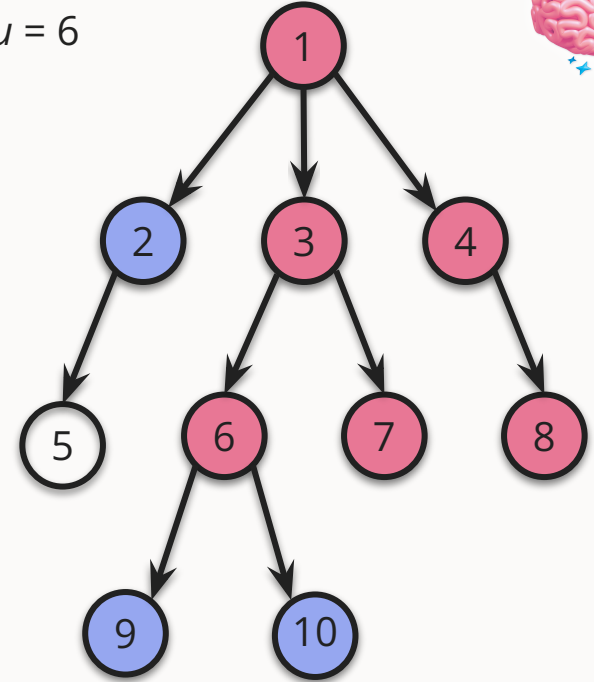
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 6$



Open: {2, 9, 10}

Closed: {1, 4, 8, 3, 7, 6}

9

2

STACK

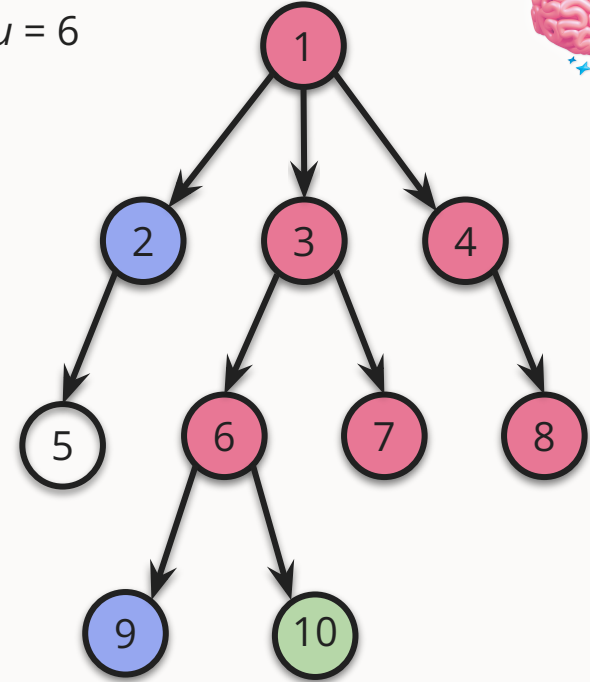
Input: El árbol de la figura

Output: Queremos retornar el nodo 10

```

1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta Sinit a Open
4  parent(Sinit) = null
5  while Open ≠ ∅
6      u ← Extraer(Open)
7      Inserta u en Closed
8      for each v ∈ Succ(u) \ (Open ∪
9  Closed)
10         parent(v) = u
11         if v ∈ G return v
        Inserta v a Open
    
```

$u = 6$



Open: {2, 9}

Closed: {1, 4, 8, 3, 7, 6}

9

2

STACK



IDDFS: DFS pero mejor

Profundidad Limitada

Agregamos un parámetro ℓ a DFS, es la **profundidad máxima** a la que buscará

Profundización Iterativa

1. Hacemos una búsqueda DFS con profundidad ℓ (comenzando por $\ell=1$)
2. Si no hubo éxito, se incrementa ℓ y repetimos

Con esta implementación modificada de DFS, ahora el algoritmo no solo es completo, si no que también encuentra el camino **óptimo**.

Esto es asumiendo que existe un nodo objetivo y una cantidad finita de estados



Heurísticas



Algoritmos vistos hasta ahora

**Búsqueda
Genérica**

DFS

IDDFS

BFS

¿Que tienen en común?



Algoritmos vistos hasta ahora

- Encuentran caminos más cortos en un grafo **ponderado**, es decir, **encuentran soluciones óptimas**
- ¿Qué sucede cuando el grafo de búsqueda es **MUY** grande?

**Alto consumo
de memoria**

**Exploración
innecesaria**

**Gran tiempo de
ejecución**

- Queremos que nuestra búsqueda obtenga el camino al nodo objetivo, **evitando revisar nodos innecesarios**. ¿Cómo lo logramos?



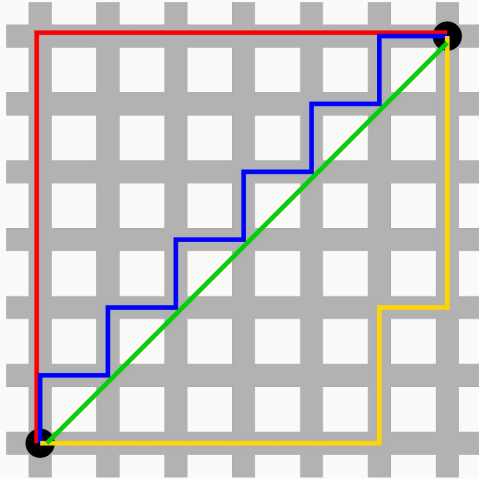
Función Heurística

- Función que **estima** la cercanía de un nodo **s** a un nodo objetivo **G**.
- Para crearla necesitamos saber **dónde se ubica el nodo objetivo** o cómo conseguirlo.
- Con ella podemos discriminar qué nodo **conviene** explorar, **priorizando** nodos que podrían llevar a la solución de forma **más rápida**.

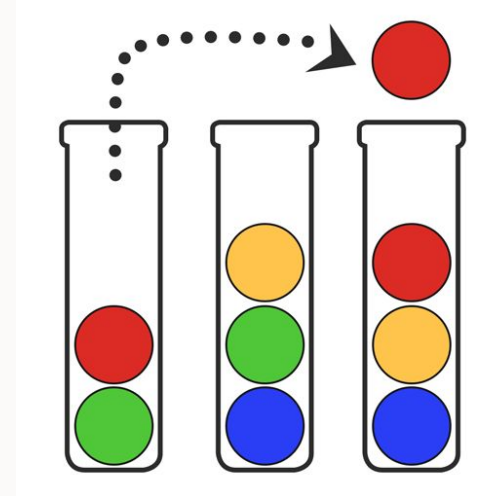
$$h(s)$$



Ejemplos de Heurísticas



- ● ● Distancia Manhattan
- Distancia Euclidiana



Cantidad de bolitas en un frasco que no tienen un mismo color



Ejemplos de Heurísticas

1	2	3
4		6
7	8	5

Cantidad de piezas en
la posición correcta

○	×	○
×		×
	○	

Cantidad de figuras del
jugador que están
alineadas



Admisibilidad

Definición

Una heurística ***h*** se dice ***admissible*** si para todo estado ***s*** se cumple que:

$$h(s) \leq h^*(s)$$

Donde $h^*(s)$ es el costo de un camino óptimo desde el estado ***s*** a un estado objetivo

Nuestra función es **admissible si nunca sobreestima** respecto a un **camino óptimo**



Admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es admisible?

H1

$H_1 = 6$				G
S				
$H_1 = 8$				

H2

$H_2 = 8$				G
S				
$H_2 = 6$				



Admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es admisible?

H1 es admisible!

$H_1 = 6$				G
S				
$H_1 = 8$				

H2 no es admisible

ya que $H_2 = 8 > h^* = 6$

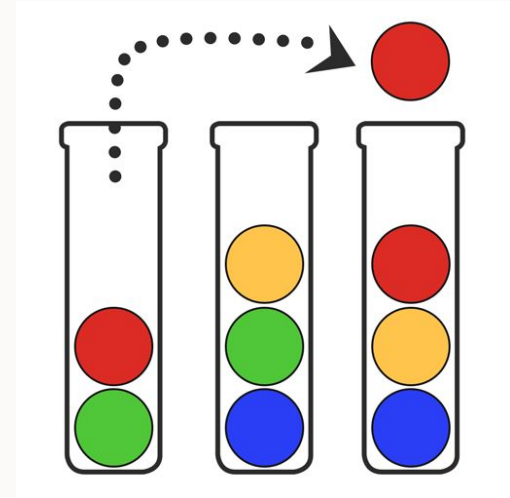
$H_2 = 8$				G
S				
$H_2 = 6$				



Admisibilidad - ejemplo

El objetivo es que cada tubo tenga bolitas de un solo color.

Una heurística admisible podría ser contar la cantidad de bolitas que no tienen el mismo color en un tubo. Notar que **al menos** este número de bolitas **se deben desplazar** para lograr el estado objetivo. Por lo que **nunca sobreestima** el trabajo que se debe realizar para llegar al objetivo.





Consistencia

Definición

Una heurística ***h*** se dice **consistente** si y sólo si:

- $h(s) = 0$, para todo $s \in G$
- $h(s) \leq c(s, s') + h(s')$ para todo vecino s' de s

O de manera equivalente $h(s) - h(s') \leq c(s, s')$

Nuestra función es consistente si nunca sobreestima en los estados objetivos ni el **costo es mayor que la variación de la heurística** entre dos estados vecinos



Consistencia

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente?

H1

$H_1 = 10$	$H_1 = 8$	$H_1 = 6$	$H_1 = 4$	$H_1 = 2$
$H_1 = 12$		$H_1 = 8$		$H_1 = 0$
S		$H_1 = 10$		$H_1 = 2$
$H_1 = 16$	$H_1 = 14$	$H_1 = 12$		$H_1 = 4$
$H_1 = 18$		$H_1 = 14$		$H_1 = 6$

G

H2

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
S		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente?

H_1 no es consistente

$H_1 = 10$	$H_1 = 8$	$H_1 = 6$	$H_1 = 4$	$H_1 = 2$
$H_1 = 12$		$H_1 = 8$		$H_1 = 0$
S		$H_1 = 10$		$H_1 = 2$
$H_1 = 16$	$H_1 = 14$	$H_1 = 12$		$H_1 = 4$
$H_1 = 18$		$H_1 = 14$		$H_1 = 6$

G

H_2 si lo es

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
S		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia y admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente y/o admisible?

$H1$

$H_1 = 5$	$H_1 = 4$	$H_1 = 3$	$H_1 = 2$	$H_1 = 1$
$H_1 = 3$		$H_1 = 2$		$H_1 = 0$
$S : H_1 = 5$		$H_1 = 3$		$H_1 = 1$
$H_1 = 6$	$H_1 = 5$	$H_1 = 4$		$H_1 = 2$
$H_1 = 7$		$H_1 = 5$		$H_1 = 3$

G

$H2$

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
$S : H_2 = 5$		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia y admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente y/o admisible?

H_1 es admisible pero no consistente

H_2 si lo es

$H_1 = 5$	$H_1 = 4$	$H_1 = 3$	$H_1 = 2$	$H_1 = 1$
$H_1 = 3$		$H_1 = 2$		$H_1 = 0$
$S : H_1 = 5$		$H_1 = 3$		$H_1 = 1$
$H_1 = 6$	$H_1 = 5$	$H_1 = 4$		$H_1 = 2$
$H_1 = 7$		$H_1 = 5$		$H_1 = 3$

G

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
$S : H_2 = 5$		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Relación consistencia y admisibilidad

Definición

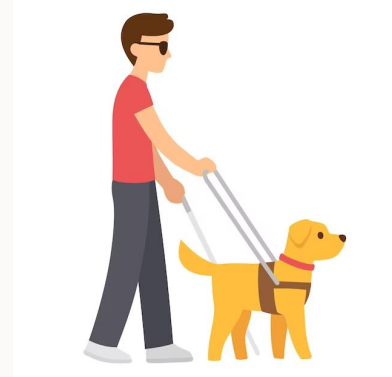
Si una heurística ***h*** es consistente, entonces ***h*** es admisible

¡Esta relación no es invertible! Es decir, si una heurística es admisible, no necesariamente va a ser consistente.



Heurísticas admisibles

- Podemos observar que una heurística admisible no es más que una estimación del costo (que **nunca sobreestima**) para llegar al estado objetivo.
- Una heurística es mejor en cuanto **mejor sea tal predicción**.
- Esta nos ayuda a discernir cuál es el mejor camino que se puede tomar dada la información del **estado actual** y a encontrar óptimos...





Dudas